

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №1
«Проектирование базы данных»

Выполнила:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Спроектировать базу данных для одного из предложенных вариантов лабораторного проекта. Для этого необходимо:

- выбрать один из предложенных вариантов работ;
- спроектировать БД, придерживаясь нотации ERD (полное соблюдение нотации не требуется, диаграмму можно выполнить в draw.io);

Выбранный вариант: вариант 1 – приложение для бронирования столиков в ресторанах.

Функциональность приложения:

- вход;
- регистрация;
- личный кабинет пользователя;
- поиск ресторанов с фильтрацией по кухне, расположению и цене;
- страница с информацией о ресторане (меню, фото, отзывы);
- история бронирований пользователя.

Дополнительно к условию в базе данных предусмотрен учёт занятых столиков и временных слотов.

Ход работы

1. Анализ требований

По требованиям к функциональности выделены три роли:

- **USER** – пользователь: регистрируется, ищет рестораны, просматривает их страницы, бронирует столики, оставляет отзывы и просматривает историю своих бронирований;
- **RESTAURANT_ADMIN** – администратор ресторана: управляет данными своего ресторана (описанием, меню, фотографиями, столиками и слотами) и подтверждает бронирования; у одного ресторана может быть несколько администраторов;
- **ADMIN** – администратор системы: управляет справочниками и всеми ресторанами.

Чтобы бронирование учитывало занятость, в базе данных выделены отдельные сущности «столик» и «временной слот»: бронирование занимает конкретный столик на конкретный слот в конкретную дату.

2. Описание сущностей

База данных состоит из 14 таблиц, три из которых – перечисления (enum). Первичным ключом каждой таблицы является поле id, внешние ключи заканчиваются на _id. Описание сущностей приведено в таблице 1.

Таблица 1 – Сущности базы данных

Сущность	Назначение и поля
User	Учётная запись пользователя. Поля: id, role (enum Roles), email (уникальный), password_hash, first_name, last_name, phone, created_at, updated_at.
Roles (enum)	Роли пользователей: ADMIN, RESTAURANT_ADMIN, USER.
City	Справочник городов: id, name. Используется для фильтрации по расположению.
Cuisine	Справочник кухонь: id, name. Используется для фильтрации по кухне.
Restaurant	Ресторан. Поля: id, city_id, cuisine_id, name, description, district, address, latitude, longitude, phone, average_check, rating, opening_time, closing_time, is_active, created_at, updated_at.
RestaurantAdmin	Связь администраторов с ресторанами («многие ко многим»): id, user_id, restaurant_id, created_at.
RestaurantPhoto	Фотографии ресторана: id, restaurant_id, url, is_main, sort_order, created_at.
MenuItem	Позиция меню ресторана: id, restaurant_id, category (enum MenuCategories), name, description, price, photo_url, is_available.
MenuCategories (enum)	Категории блюд: APPETIZER, MAIN_COURSE, DESSERT, DRINK.
Review	Отзыв пользователя о ресторане: id, restaurant_id, user_id, rating (от 1 до 5), comment, created_at, updated_at.
RestaurantTable	Столик ресторана: id, restaurant_id, number, capacity (вместимость), zone (зона зала), is_active.
TimeSlot	Временной слот ресторана – шаблон интервала в течение дня (например, 12:00–14:00): id, restaurant_id, start_time, end_time, is_active.
Booking	Бронирование столика: id, user_id, restaurant_table_id, time_slot_id, status (enum BookingStatuses), booking_date, guests_count, comment, created_at, updated_at.
BookingStatuses (enum)	Статусы бронирования: PENDING, CONFIRMED, CANCELLED, COMPLETED.

3. ERD-диаграмма

Диаграмма draw.io в нотации ERD и приведена на рисунке 1.

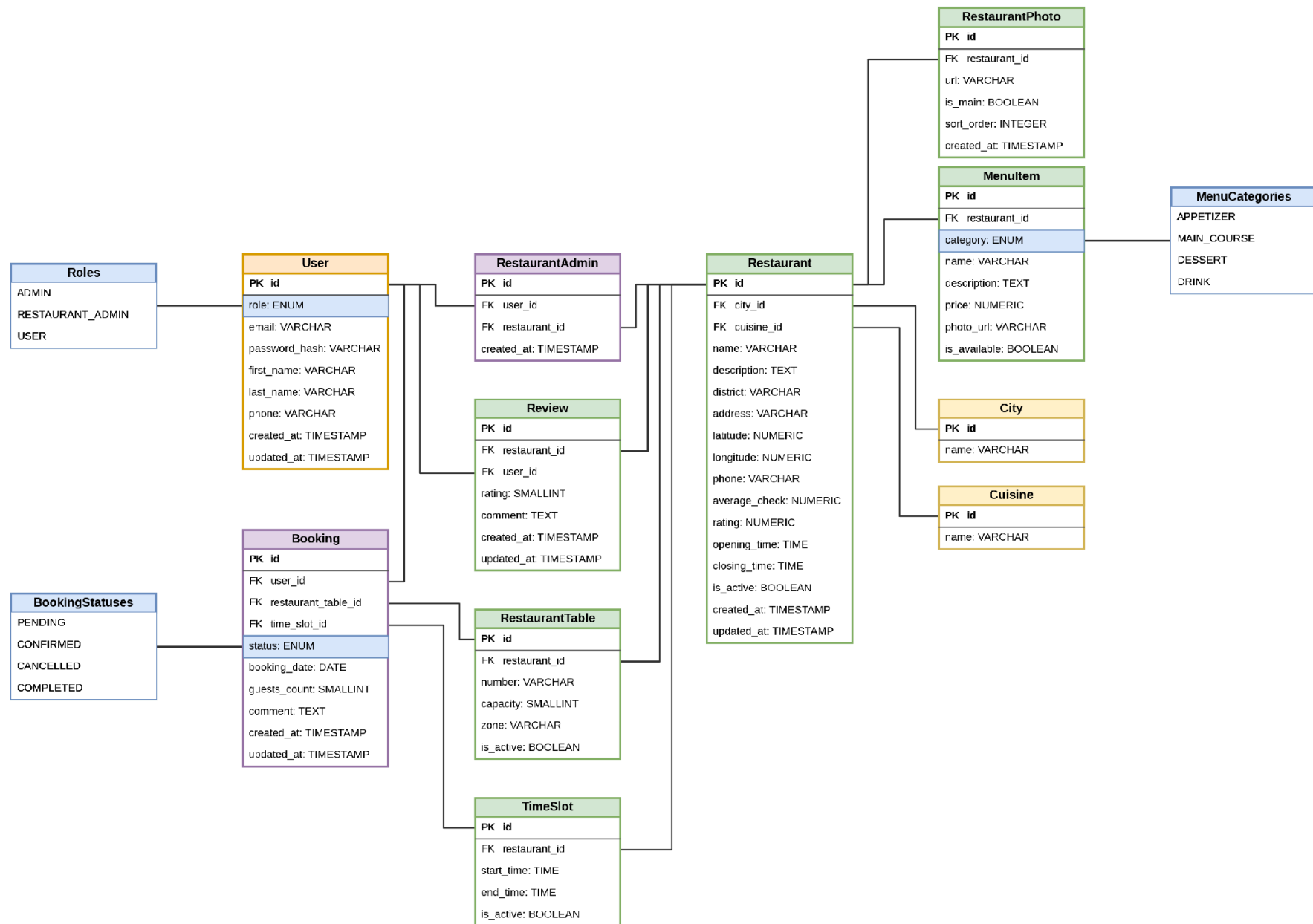


Рисунок 1 – ERD-диаграмма базы данных приложения для бронирования столиков

Обозначения: оранжевый – пользователи; зелёный – основные сущности; жёлтый – справочники; фиолетовый – связующие таблицы; синий – перечисления (enum); PK – первичный ключ, FK – внешний ключ.

4. Связи между сущностями

Связи между сущностями и способ их реализации приведены в таблице 2. Связь ресторана с бронированиями косвенная: она проходит через столики (RestaurantTable) и слоты (TimeSlot) ресторана.

Таблица 2 – Связи между сущностями

Связь	Тип	Реализация
User – Roles	N:1	поле User.role (enum)
User – Booking	1:N	Booking.user_id
User – Review	1:N	Review.user_id
User – Restaurant	M:N	таблица RestaurantAdmin (user_id, restaurant_id)
Restaurant – City	N:1	Restaurant.city_id
Restaurant – Cuisine	N:1	Restaurant.cuisine_id
Restaurant – RestaurantPhoto	1:N	RestaurantPhoto.restaurant_id
Restaurant – MenuItem	1:N	MenuItem.restaurant_id
Restaurant – Review	1:N	Review.restaurant_id
Restaurant – RestaurantTable	1:N	RestaurantTable.restaurant_id
Restaurant – TimeSlot	1:N	TimeSlot.restaurant_id
RestaurantTable – Booking	1:N	Booking.restaurant_table_id
TimeSlot – Booking	1:N	Booking.time_slot_id
Booking – BookingStatuses	N:1	поле Booking.status (enum)
MenuItem – MenuCategories	N:1	поле MenuItem.category (enum)

5. Учёт занятых столиков и слотов

Для учёта занятости используются три таблицы:

- **RestaurantTable** описывает физические столики ресторана: номер, вместимость (capacity) и зону зала;
- **TimeSlot** задаёт шаблон временных интервалов ресторана (например, 12:00–14:00, 14:00–16:00), одинаковый для всех дат, поэтому отдельные строки на каждый день хранить не нужно;
- **Booking** занимает конкретный столик на конкретный слот в конкретную дату: поля restaurant_table_id, time_slot_id и booking_date.

Занятость столика однозначно определяется тройкой «столлик – слот – дата». Чтобы исключить двойное бронирование, для таблицы Booking задаётся уникальный индекс по этим трём полям, действующий для активных статусов (PENDING и CONFIRMED). Отмененная бронь (CANCELLED) освобождает столик, при этом запись остаётся в истории пользователя.

При создании брони дополнительно проверяется, что guests_count не превышает capacity выбранного столика, а дата бронирования не находится в прошлом. История бронирований пользователя формируется выборкой из Booking по user_id с сортировкой по booking_date.

6. Проектные решения и ограничения

- **Хранение паролей.** В таблице User хранится не пароль, а его хэш (password_hash). При входе введенный пароль сравнивается с хэшем.
- **Restaurant.rating хранится в таблице (денормализация).** Значение можно вычислить как среднее по Review.rating, однако поиск ресторанов должен быстро сортироваться и фильтроваться по рейтингу без соединения и группировки по таблице отзывов. Поле пересчитывается при создании, изменении и удалении отзыва (в одной транзакции с сохранением отзыва), источником данных остаётся таблица Review.
- **Restaurant.average_check хранится, так как его нельзя вычислить.** Это данные о ресторане, которые вводит администратор: средний чек на гостя не равен средней цене блюд из меню, а данных о заказах в системе нет. Поле используется в фильтре по цене.
- **Меню хранится только как набор позиций MenuItem.** Отдельной таблицы Menu нет: у ресторана одно меню, то есть связь была бы 1:1, а такая таблица содержала бы только идентификатор и restaurant_id. Меню ресторана определяется полем MenuItem.restaurant_id. Если в будущем потребуется несколько меню (основное, барное, детское), добавляется таблица Menu, а MenuItem получает внешний ключ menu_id.
- **Администраторы ресторана.** Так как у ресторана может быть несколько администраторов, связь User и Restaurant реализована

через таблицу RestaurantAdmin. Администратор ресторана управляет только теми ресторанами, для которых есть запись в этой таблице.

- **Перечисления.** Роли, статусы бронирования и категории блюд вынесены в enum (Roles, BookingStatuses, MenuCategories), так как наборы значений фиксированы.
- **Расположение и поиск.** Расположение ресторана задаётся городом (City), районом (district) и адресом; координаты (latitude, longitude) позволяют показать ресторан на карте, если в будущем появится такая необходимость. Для ускорения поиска планируются индексы по полям cuisine_id, city_id и average_check.

7. Ограничения целостности и поведение при удалении

Ограничения, которые задаются в базе данных, приведены в таблице 3. Первичные и внешние ключи в таблицу не включены: они обязательны для всех таблиц.

Таблица 3 – Ограничения целостности

Сущность	Поле (поля)	Ограничение
User	email	UNIQUE, NOT NULL
User	password_hash, first_name, last_name	NOT NULL
User	role	NOT NULL, по умолчанию USER
User	phone	необязательное поле (допускается NULL)
City	name	UNIQUE, NOT NULL
Cuisine	name	UNIQUE, NOT NULL
Restaurant	name, city_id, cuisine_id, opening_time, closing_time	NOT NULL
Restaurant	average_check	CHECK: значение не меньше 0
Restaurant	rating	CHECK: от 0 до 5, по умолчанию 0
Restaurant	is_active	по умолчанию true
Restaurant	latitude, longitude	необязательные; CHECK: latitude от -90 до 90, longitude от -180 до 180
RestaurantAdmin	(user_id, restaurant_id)	UNIQUE – пользователь не назначается администратором ресторана дважды
RestaurantPhoto	url	NOT NULL

Сущность	Поле (поля)	Ограничение
RestaurantPhoto	restaurant_id, is_main	не более одного главного фото на ресторан (частичный UNIQUE по restaurant_id при is_main = true)
MenuItem	name, category	NOT NULL
MenuItem	price	NOT NULL, CHECK: значение не меньше 0
MenuItem	is_available	по умолчанию true
Review	(user_id, restaurant_id)	UNIQUE – один отзыв пользователя на ресторан
Review	rating	NOT NULL, CHECK: от 1 до 5
RestaurantTable	(restaurant_id, number)	UNIQUE – номера столиков не повторяются в одном ресторане
RestaurantTable	number, capacity	NOT NULL; для capacity CHECK: значение больше 0
TimeSlot	(restaurant_id, start_time)	UNIQUE – в ресторане нет двух слотов с одним началом
TimeSlot	start_time, end_time	NOT NULL, CHECK: start_time меньше end_time
Booking	(restaurant_table_id, time_slot_id, booking_date)	UNIQUE только для статусов PENDING и CONFIRMED (частичный индекс)
Booking	guests_count	NOT NULL, CHECK: значение больше 0
Booking	status	NOT NULL, по умолчанию PENDING
Booking	booking_date	NOT NULL

Общие правила

- все внешние ключи объявляются как NOT NULL;
- поля created_at по умолчанию получают текущее время (now()), а updated_at обновляется при каждом изменении записи;
- перечисления (Roles, BookingStatuses, MenuCategories) хранятся как enum, поэтому допустимые значения проверяются самой базой данных.

Проверки на уровне приложения. Часть правил нельзя выразить ограничением в базе данных, поэтому они проверяются в коде бэкенда:

- guests_count не превышает capacity выбранного столика (сравниваются поля из разных таблиц);
- дата бронирования не находится в прошлом (ограничение CHECK с текущей датой не остаётся истинным со временем);

- слоты одного ресторана не пересекаются по времени;

Поведение при удалении (ON DELETE). Данные, которые не имеют смысла без родителя (фотографии, меню, столики, слоты, назначения администраторов), удаляются вместе с рестораном (CASCADE). Данные, составляющие историю (бронирования и отзывы), каскадно не удаляются: удаление родительской записи запрещается (RESTRICT). Настройки для всех внешних ключей приведены в таблице 4.

Таблица 4 – Поведение при удалении записей

Внешний ключ	ON DELETE	Пояснение
Restaurant.city_id → City	RESTRICT	нельзя удалить город, в котором есть рестораны
Restaurant.cuisine_id → Cuisine	RESTRICT	нельзя удалить кухню, которая указана у ресторанов
RestaurantAdmin.user_id → User	CASCADE	при удалении пользователя его назначения удаляются
RestaurantAdmin.restaurant_id → Restaurant	CASCADE	при удалении ресторана удаляются назначения администраторов
RestaurantPhoto.restaurant_id → Restaurant	CASCADE	фотографии не нужны без ресторана
MenuItem.restaurant_id → Restaurant	CASCADE	меню не нужно без ресторана
RestaurantTable.restaurant_id → Restaurant	CASCADE	столики не нужны без ресторана
TimeSlot.restaurant_id → Restaurant	CASCADE	слоты не нужны без ресторана
Review.restaurant_id → Restaurant	RESTRICT	отзывы не удаляются вместе с рестораном
Review.user_id → User	RESTRICT	отзывы не удаляются вместе с пользователем
Booking.user_id → User	RESTRICT	нельзя удалить пользователя с историей бронирований
Booking.restaurant_table_id → RestaurantTable	RESTRICT	нельзя удалить столик, на который есть брони
Booking.time_slot_id → TimeSlot	RESTRICT	нельзя удалить слот, на который есть брони

Так как столики ресторана связаны с бронированиями через RESTRICT, ресторан с историей бронирований удалить нельзя, даже если остальные его данные удаляются каскадно. Вместо удаления используется

деактивация: Restaurant.is_active, RestaurantTable.is_active, TimeSlot.is_active и MenuItem.is_available устанавливаются в false. Удаление пользователей с бронированиями и отзывами не предусмотрено.

8. Соответствие функциональным требованиям

Таблицы, обеспечивающие каждое требование, приведены в таблице 5.

Таблица 5 – Соответствие требований и таблиц БД

Требование	Таблицы
Вход и регистрация	User (email, password_hash, role)
Личный кабинет пользователя	User, Booking, Review
Поиск ресторанов с фильтрацией по кухне, расположению и цене	Restaurant (cuisine_id, city_id, district, average_check), Cuisine, City
Страница ресторана (меню, фото, отзывы)	Restaurant, MenuItem, RestaurantPhoto, Review
История бронирований пользователя	Booking (user_id), RestaurantTable, TimeSlot
Учёт занятых столиков и слотов	RestaurantTable, TimeSlot, Booking
Управление рестораном администратором ресторана	RestaurantAdmin, Restaurant, MenuItem, RestaurantPhoto, RestaurantTable, TimeSlot

Вывод

В качестве темы выбран вариант 1. Разработана ERD-диаграмма из 14 сущностей, описаны связи и ограничения целостности. Описаны учет занятых столиков и временных слотов, а также разграничение ролей пользователя, администратора ресторана и администратора системы. Полученная модель будет использована в следующих работах: для проектирования API в формате OpenAPI (Д32) и для реализации моделей, представлений и контроллеров (ЛР1).